Section 11.6 of 10: Agent-to-Agent (A2A) Protocol

Executive Overview

A2A enables agent collaboration across vendor and organizational boundaries. Where MCP handles agent-to-tool integration (structured, synchronous, within a trust boundary), A2A handles agent-to-agent delegation (asynchronous, cross-org, peer-to-peer). Understanding task lifecycles, capability discovery via Agent Cards, and failure recovery is essential for designing robust multi-agent systems.

11.6.1 MCP vs. A2A — When to Use Each

Protocol Comparison

```
MCP (Agent → Tool):                    A2A (Agent → Agent):
──────────────────────────────────     ──────────────────────────────────
Agent calls tool/function              Agent delegates task to peer agent
Host mediates all calls                Peer-to-peer (no shared host)
Synchronous request/response           Asynchronous task lifecycle
Same trust boundary                    Cross-org, federated trust
Tool knows nothing about agent         Agents negotiate capabilities
Returns structured data                Returns completed artifacts
```

When to Use Which

Scenario	Use MCP	Use A2A
Querying a database	✓	
Calling a weather API	✓	
Delegating research to a specialist agent at another company		✓
Running a code interpreter locally	✓	
Orchestrating a multi-step workflow across autonomous agents		✓
Getting structured data from a file system	✓	
Commissioning a translation agent you don't own		✓

Key insight: MCP is a function call interface. A2A is an async job delegation interface. If you'd use a REST call to a microservice, MCP fits. If you'd use a job queue with a remote contractor, A2A fits.

11.6.2 Agent Cards — Capability Discovery

Agent Card Schema

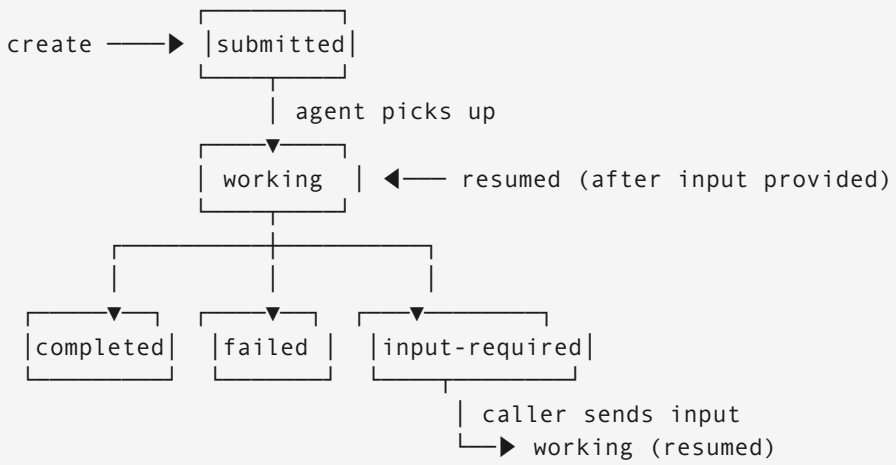
```
{
  "name": "Research Agent",
  "description": "Performs deep research on scientific and technical topics",
  "url": "https://agents.example.com/research",
  "version": "1.2.0",
  "skills": [
    {
      "name": "literature_search",
      "description": "Search academic papers and return structured summaries",
      "input_modes": ["text/plain"],
      "output_modes": ["application/json", "text/markdown"],
      "estimated_duration_seconds": 120,
      "pricing": {"per_task": 0.05, "currency": "USD"}
    },
    {
      "name": "web_research",
      "description": "Synthesize information from web sources on a topic",
      "input_modes": ["text/plain"],
      "output_modes": ["text/markdown"],
      "estimated_duration_seconds": 60
    }
  ],
  "authentication": {
    "schemes": ["bearer", "oauth2"],
    "oauth2_endpoint": "https://auth.example.com/token"
  },
  "rate_limits": {"requests_per_minute": 10}
}
```

Discovery Pattern

```
Orchestrator needs "academic research" capability:
  1. Query agent registry: GET /registry?skill=academic_research
  2. Receive list of Agent Cards matching capability
  3. Select based on: trust score, latency SLA, cost, output format
  4. Cache Agent Card for session duration
  5. Re-fetch if card version changes (ETags)
```

11.6.3 Task Lifecycle State Machine

State Transitions




```
Also: any state → canceled (by caller)
```

Task Operations

```
# Create task
task = a2a_client.create_task({
    "skill": "literature_search",
    "params": {"topic": "transformer attention mechanisms", "max_papers": 10},
    "deadline": datetime.now() + timedelta(seconds=300)
})

# Poll or subscribe to status
async for event in a2a_client.stream_events(task.id):
    if event.state == "input-required":
        a2a_client.send_input(task.id, "Focus on post-2022 papers only")
    elif event.state == "completed":
        artifacts = a2a_client.get_artifacts(task.id)
        break
    elif event.state == "failed":
        handle_failure(event.error)
        break
```

Streaming Updates via SSE

```
GET /tasks/{task_id}/events
Accept: text/event-stream

data: {"state": "working", "progress": "25%", "message": "Searching arXiv..."}
```

```
data: {"state": "working", "progress": "60%", "message": "Found 23 papers,
filtering..."}
data: {"state": "working", "progress": "90%", "message": "Synthesizing
summaries..."}
data: {"state": "completed", "artifact_count": 2}
```

11.6.4 Communication Object Model

Messages and Parts

Messages can carry heterogeneous content using typed parts:

```
message = {
    "role": "agent",
    "parts": [
        {"kind": "text", "text": "I need clarification on the scope."},
        {"kind": "data", "data": {"ambiguous_terms": ["recent",
"comprehensive"]}},
        {"kind": "file", "file": {"uri": "file:///partial_results.json"}}
    ]
}
```

Artifacts

Task outputs are structured artifacts with metadata:

```
{
  "artifacts": [
    {
      "kind": "text",
      "name": "research_summary",
      "text": "## Research Synthesis\n...",
      "metadata": {
        "confidence": 0.88,
        "sources": 14,
        "generated_at": "2025-06-09T12:00:00Z"
      }
    },
    {
      "kind": "file",
      "name": "full_bibliography",
      "file": {"uri": "https://storage.example.com/results/bib_123.json"},
      "metadata": {"record_count": 14}
    }
```

```
  ]
}
```

Interview Problems

Problem 1: Supervisor Agent Orchestrating Specialist Agents with Failure Handling

Scenario: A supervisor delegates research tasks to 3 specialist agents (web, academic, news). One agent times out. How do you handle it?

Solution:

```
async def coordinate_research(topic: str, deadline_sec: int = 300):
    # Discover agents
    agents = {
        "web":      find_agent("web_search"),
        "academic": find_agent("literature_search"),
        "news":     find_agent("news_search")
    }

    # Create tasks with per-agent deadlines
    tasks = {
        name: create_task(agent, topic, deadline=deadline_sec - 30)
        for name, agent in agents.items()
    }

    # Gather with individual timeouts
    results = {}
    errors = {}

    async def wait_for_task(name, task_id):
        try:
            result = await asyncio.wait_for(
                poll_until_done(task_id),
                timeout=deadline_sec - 30
            )
            results[name] = result
        except asyncio.TimeoutError:
            await cancel_task(task_id)
            errors[name] = "timeout"
        except AgentError as e:
            errors[name] = str(e)

    await asyncio.gather(*[
        wait_for_task(name, task.id)
        for name, task in tasks.items()
    ])
```

```
    # Partial result synthesis
    if not results:
        raise AllAgentsFailed("No results available")

    synthesis = synthesize(results)

    if errors:
        synthesis.metadata["degraded_sources"] = list(errors.keys())
        synthesis.metadata["confidence"] = 0.6  # Flag lower confidence

    return synthesis
```

Key design choices: - Each agent gets its own deadline (total - 30s buffer for synthesis) - Partial results are used; complete failure is the exception, not the norm - Metadata annotates which sources were unavailable (caller can decide to retry)

Problem 2: Handling input-required Mid-Task

Scenario: A research agent sends back `input-required` partway through a long task (e.g., it needs clarification on scope). The user is not available for 10 minutes. Design the interaction.

Solution:

```
Flow:

1. Task enters input-required state
   Agent message: "Found 3 interpretations of 'recent AI papers':
     (a) last 6 months, (b) last 2 years, (c) post-GPT-3 era.
     Which scope do you mean?"

2. Supervisor receives input-required event
   Options:
     a) Route to user if available (real-time)
     b) Apply default heuristic: "recent" = last 12 months
     c) Continue with all interpretations and let user choose from results

3. Supervisor sends automated input (heuristic b):
   a2a_client.send_input(task.id, {
       "clarification": "Use last 12 months as 'recent'",
       "automated": True
   })

4. Task resumes from working state

5. Supervisor logs automated decision for audit trail

Design principle:
  Prefer automated defaults over blocking indefinitely.
```

```
If automated response produces wrong results, the cost is lower
than leaving the task frozen for 10 minutes.
Always log automated decisions for human review.
```

Problem 3: Security at Agent Delegation Boundaries

Scenario: Your orchestrator delegates sensitive customer analysis to an external agent. How do you ensure it cannot exfiltrate data?

Solution:

```
Security controls at each delegation boundary:

1. Minimal data principle:
   Send only what the agent needs
   Bad:  send_task(agent, {"customer_record": full_customer_dict})
   Good: send_task(agent, {"anonymized_id": hash_id, "transactions": count_only})

2. Output validation:
   Before accepting artifact:
   - Scan for PII patterns (SSN, credit cards, names)
   - Check for unusual data volumes (unexpected exfiltration signal)
   - Verify output matches expected schema

3. Task sandboxing:
   - Delegate agent has no network access to your internal systems
   - Only receives what you explicitly pass as input
   - Cannot call back into your MCP servers

4. Time-limited credentials:
   - Any tokens passed expire within task deadline + 5 minutes
   - No long-lived credentials ever passed via A2A

5. Audit every delegation:
   {
     "task_id": "...",
     "agent_url": "https://agents.external.com/analysis",
     "agent_card_hash": "sha256:abc...",  # Pin to known card version
     "data_sent_hash": "sha256:xyz...",   # What you sent
     "result_hash": "sha256:def..."       # What you received
   }
```